SWNA Workshop

Facilitator Guide

Session Reference & Troubleshooting

For the session facilitator — 2026

1. Pre-Session Checklist

- [ ] Package installable: test `pip install whitenoise-swna` or GitHub install on a fresh environment
- [ ] Sample datasets accessible: `earthquake_ph.csv`, `sunspot_monthly.csv`, `dna_nucleotide.csv`
- [ ] Colab notebook tested end-to-end (all cells run without error)
- [ ] Google Drive or USB backup of all sample files
- [ ] Participant guide distributed (printed or shared digitally)
- [ ] Projector / screen tested
- [ ] Participant guide link ready to paste into group chat

2. Session Timing Guide

Time	Activity	Facilitator notes
0 – 10 min	Setup & install	Help stragglers with pip install; verify imports work
10 – 20 min	CSV format & `read_csv()`	Show V1–V4 warnings live with a bad CSV
20 – 35 min	Pipeline Steps A–F (earthquake data)	Walk through slowly; check everyone sees the MSD plot
35 – 45 min	Model selection & R^2	Explain μ regimes table; demonstrate trying a wrong model
45 – 55 min	Compare & batch	Use both earthquake + sunspot datasets
55 – 65 min	DNA model demo	Highlight restricted diffusion; note no μ for DNA model
65 – 80 min	Participants' own data	Roam and assist individually
80 – 90 min	Q&A, export results	Show `publish_msd()` and `to_csv()`

3. Complete Function Reference

I/O

```
wn.read_csv(path: str) → (time, values, meta)
```

- Reads 2-column CSV with `name [unit]` headers
- Returns: `time` (1D array), `values` (1D array), `meta` (dict)
- Meta keys: `x_name`, `x_unit`, `x_label`, `y_name`, `y_unit`, `y_label`, `source_file`, `n_points`
- Raises `FileNotFoundError` (✗ prefix) or `ValueError` for bad format
- Prints ⚠ warnings V1–V4 (non-fatal)

```
wn.read_csv_multi(path: str) → list of (time, values, meta)
```

- Multi-column CSV; column 1 = time, each other column = one dataset

Preprocessing

```
wn.detrend(values, method='linear', poly_order=1) → array
```

- methods: `'linear'`, `'polynomial'`, `'mean'`

```
wn.normalize(values, method='zscore') → array
```

- methods: `'zscore'`, `'minmax'`, `'mean'`, `'maxabs'`

```
wn.smooth(values, window=5, method='moving_average') → array
```

- methods: `'moving_average'`, `'gaussian'`

Core

```
wn.compute_msd(x, max_lag=None, normalize=False) → (lags, msd)
```

- Default `max_lag = N // 2` (hard-capped at N/2)
- `normalize=True` divides all MSD values by MSD[lag=1]

```
wn.fit_msd(lags, msd, model='cosine', p0=None, bounds=None, max_lag_fraction=1.0) → FitResult |
None
```

```
wn.list_models() → None # prints formatted table of all 16+ models
```

```
wn.get_model(name: str) → dict
```

Analysis

```
wn.analyze(source, model='cosine', label='', max_lag=None, max_lag_fraction=1.0, verbose=True)
→ AnalysisResult
```

- `source`: CSV path string OR numpy array (array requires `label=`)
- Returns `AnalysisResult` with: `.summary()`, `.fit`, `.regime`, `.mu`, `.to_csv()`, `.values`, `.lags`

```
wn.compare(datasets, model='cosine') → ComparisonResult
```

- `datasets`: dict `{label: path_or_array}` or list of paths
- Per-dataset models: `model={'A': 'cosine', 'B': 'exponential'}`

```
wn.batch_analyze(source, model='cosine', pattern='*.csv') → ComparisonResult
```

- `source`: folder path string, list of paths, or multi-column CSV path

Exploratory Plots (viz/explore.py)

All accept `ax=None, show=True` and return a `Figure`.

Function	Signature
`plot_series`	`(time, values, metadata=None, title='', ax=None, show=True)`
`plot_msd_empirical`	`(lags, msd, metadata=None, title='', ax=None, show=True)`
`plot_msd`	`(result, ax=None, show=True)`
`plot_pdf`	`(result, lag_index=None, ax=None, show=True)`
`plot_timeseries`	`(result, ax=None, show=True)`
`plot_diagnostics`	`(result, show=True)` — 2×2 figure

Publication Plots (viz/publish.py)

Function	Signature
`publish_msd`	`(result, palette='default', figsize=(5,4), save_path=None, show=True)`
`publish_pdf`	`(result, lag_index=None, palette='default', figsize=(5,4), save_path=None, show=True)`
`publish_comparison`	`(cr, palette='default', figsize=None, save_path=None, show=True)`

Palettes: `'default'` or `'colorblind'`. Save as PDF: `save_path='figure.pdf'`.

4. Models Reference

Model	Params	MSD formula	Notes
`exponential`	μ, β, N	$\Gamma(\mu)\cdot\beta^{-\mu}\cdot T^{\mu-1}\cdot e^{-\beta/T}$	Most stable; good default starting point
`cosine`	μ, ν, N	Power-law × cos(νT/2) × Bessel $J_{\mu-½}$	Oscillatory; valid range T < π/ν
`sine`	μ, ν, N	Same as cosine with sin(νT/2)	Wider valid range T < 2π/ν; MSD=0 at T=0
`fbm`	H, N	T^{2H}	H=0.5 → Brownian; reports H not μ
`dna`	a, b, c, N	$N\cdot(a - c\cdot e^{-bL})$	Restricted diffusion; no μ or regime
12 stubs	—	NotImplementedError	Not yet implemented; use above 5

> **Facilitator tip:** Start participants with `'exponential'` — it converges reliably for most inter-event time datasets. Introduce `'cosine'` only after they've seen a successful fit.

5. Memory Parameter μ — Full Regime Table

μ range	Regime	Plain-language description
μ < 0.95	SUBDIFFUSIVE	Past fluctuations resist change — slow spreading
0.95 – 1.05	NEAR-BROWNIAN	Weak or no memory — near-ordinary random walk
1.05 – 2.0	SUPERDIFFUSIVE	Past fluctuations reinforce future ones — fast spreading
μ > 2.0	HYPERBALLISTIC	Extreme memory — strongly persistent dynamics

For `fbm` model: H < 0.475 → subdiffusive; 0.475–0.525 → near-Brownian; H > 0.525 → superdiffusive.

6. Validated Benchmarks

Use these to verify participant results look reasonable:

Dataset	Model	Expected result	Reference
Philippine earthquakes	exponential	μ ≈ 1.00 – 1.19	Roque et al. 2024
Solar sunspots	exponential	μ ≈ 1.15	Toledo et al. 2024
CO_2 Keeling Curve	cosine	μ ≈ 0.91 – 0.97	Elnar et al. 2024
DNA nucleotide	dna	a ≈ 5.21, b ≈ 0.0024, c ≈ 3.81	Violanda et al. 2019
X-ray binaries	cosine	μ ∈ [0.50, 1.39]	Calotes thesis 2024

7. Troubleshooting Guide

Symptom	Likely cause	Fix
`FileNotFoundError`	Wrong path or working directory	Check current directory with `import os; os.getcwd()`; use relative path from notebook location
`AttributeError: 'plot_msd_raw'`	Outdated notebook cell	Use `wn.plot_msd_empirical()` (not `plot_msd_raw`)
`TypeError: plot_series() got unexpected keyword 'xlabel'`	Old notebook cell	Use `metadata=meta` instead of `xlabel=`
$R^2 < 0.3$	Wrong model for the data	Try a different model; run `wn.list_models()`
`N = 0.0000` in fit summary	Data amplitude too small (optimizer drives N → 0)	Normalize first: `scaled = wn.normalize(values, 'zscore')` then `wn.analyze(scaled, ...)`
PDF histogram much narrower than Gaussian	Leptokurtic / intermittent data (bursty fluctuations)	Physically meaningful — not a bug. Try a larger `lag_index=` value.
⚠ "Time column not strictly increasing"	Data not sorted by time	Sort by column 1 before saving the CSV
`NotImplementedError` for chosen model	Stub model selected	Use one of: `cosine`, `exponential`, `sine`, `fbm`, `dna`
Blank rows / NaN error from CSV	Raw data export with empty lines	`pd.read_csv(f, skip_blank_lines=True).dropna(how='all').to_csv('clean.csv', index=False)`
No plot appears in Colab	Backend issue	Add `%matplotlib inline` before imports; or `import matplotlib; matplotlib.use('Agg')`

8. DNA Model — Facilitator Notes

> **Key difference:** The `'dna'` model does *not* extract a memory parameter μ. It fits a **restricted diffusion / plateau** model: MSD saturates to a finite value at large lag. There is no regime classification (subdiffusive / superdiffusive) for this model.

MSD formula: $MSD(L) = N \cdot (a - c \cdot e^{-bL})$

Parameter	Meaning	Published value
`a`	Plateau MSD at large lag	≈ 5.21
`b`	Rate at which plateau is reached	≈ 0.0024
`c`	Exponential amplitude (must satisfy c < a)	≈ 3.81
`N`	Normalization scalar	≈ 1.0

Facilitator tip: The plateau is only visible for large lags ($L \gg 1/b \approx 400$). The sample dataset has n=2000 points (max_lag=1000) specifically to make the plateau visible. If using a shorter dataset, the plateau may not appear and the fit may perform poorly.

9. CSV Format — Full Specification

```
name1 [unit1], name2 [unit2]
value, value
value, value
```

Rule	Detail
Header required	Row 1 is always the header. No exceptions.
Header format	`name [unit]` — space before bracket optional
Column 1	Always the independent variable (time, index, position…)
Column 2	Always the observable (what was measured)
Unitless	Use empty brackets: `flux []`
Multi-column	Supported for batch: time + N observables → N separate analyses

Validation Warnings (V1–V4) — all non-fatal

- **V1:** Time not strictly increasing → "⚠ Time column is not strictly increasing."
- **V2:** Negative values in count/number columns → "⚠ Negative values found…"
- **V3:** Time column values too large (possible column swap) → "⚠ Time column values look unusually large."
- **V4:** Unrecognized time unit → "⚠ Unrecognized time unit '…'."

10. Quick Command Reference

Function	Purpose
`wn.read_csv(path)`	Load 2-column CSV → (time, values, meta)
`wn.read_csv_multi(path)`	Load multi-column CSV → list of tuples
`wn.plot_series(time, values, metadata=meta)`	Plot raw series
`wn.compute_msd(values)`	Compute MSD → (lags, msd)
`wn.plot_msd_empirical(lags, msd, metadata=meta)`	Plot MSD scatter
`wn.analyze(path, model='exponential')`	Full pipeline → AnalysisResult
`result.summary()`	Print μ, R^2, regime
`wn.plot_diagnostics(result)`	2×2 diagnostic figure
`wn.compare({...}, model='...')`	Compare datasets → ComparisonResult
`wn.batch_analyze(folder, model='...')`	Batch folder → ComparisonResult
`wn.list_models()`	Print all 16+ models and status
`result.to_csv(path)`	Export result to CSV
`comp.to_csv(path)`	Export comparison table to CSV
`wn.publish_msd(result, save_path='fig.pdf')`	Publication MSD figure
`wn.publish_comparison(comp, save_path='fig.pdf')`	μ bar chart
`wn.detrend(values, method='linear')`	Remove linear trend
`wn.normalize(values, method='zscore')`	Normalize (fix N=0 issue)

SWNA Workshop Facilitator Guide · 2026 · Keep this page for your reference